DATE: 10/15/2026
TO: Dr Kid Kidd
FROM: Paa Woode
SUBJECT: Project 3

---

1 INTRODUCTION

As engineers, the documentation of a project is just as crucial as implementing the project itself. The purpose of this project is to modify the design documented in Project 2 so that the Arduino Millis() function, together with the register definitions and interrupt service routine (ISR) behind it, is completely replaced by a function written by the Developer, named mymilli(), as directed by Prof. Kidd. The Customer Requirements and High-level Technical Requirements are unchanged. Some Educational Constraints are changed. This document is a new revision of project 2. It includes an Engineering Change Note (ECN2.1) in Section 8 and marks all changes to project 2 with the date of the change

2 PROBLEM STATEMENT

There are 3 parts to this assignment, mirroring project 1 and project 2. In Part I, the Customer Requirements and High-level Engineering Requirements are unchanged and are incorporated by reference, and the changed Educational Constraints are listed. In Part II, the testing requirements of project 1 and project 2 are retained and extended with the tests needed to evaluate mymilli(). In Part III, the system is redesigned so that its timekeeping is provided by a Developer-written timer ISR and register constants, then built and tested. ECO 0.3 states the reasons for the change: a design the team can customize and extend, a design isolated from future changes in the third-party arduino.h, and avoidance of possible future intellectual property ownership claims.

3 EQUIPMENT

- Arduino UNO R3
- 2 LEDs
- 2 Resistors
- 10 kΩ resistors
- Arduino IDE 2.3.3 or better, a computer, and a USB 2.0 type A to type B cable.

4 PART I CUSTOMER REQUIREMENTS -> HIGH-LEVEL ENGINEERING REQUIREMENTS

CUSTOMER REQUIREMENTS: The customer requirements, C1 through C6, are identical to those of project 1. See project 1 Section 4, Table 1.

HIGH-LEVEL TECHNICAL REQUIREMENTS: The high-level technical requirements, HL.1 through HL.6, are identical to those of PROJECT-ASYNC and PROJECT-CE. See project 1 Section 5, Table 2.

EDUCATIONAL CONSTRAINTS: ECO 0.3 project 3 changes the Educational Constraints as follows. The labels EC.1.0 through EC.1.3 are those used in this project. They are distinct from EC.1 of project 1 (use of the AVR gcc), which continues to apply, and the constraints EC.2 through EC.5 of project 1 are unchanged. See project 1 Section 5, Table 3.

EC.1.0	The Developer must replace the millis() function provided by arduino.h with the Developer's own function, named mymilli(). The replacement may extend the capability of millis().
EC.1.0.1	The Developer must write their own PWM counter ISR and use it as the timing mechanism underlying mymilli().
EC.1.0.2	The Developer must define their own register constants that would normally be defined in avr.h. They need not have the same names.
EC.1.1	The testing and validation requirements must contain those defined in PROJECT-CE.
EC.1.1.1	The testing and validation requirements must be extended with any additional tests necessary to evaluate the modified system.
EC.1.2	The final report must be a new revision of the formal design document produced in PROJECT-CE, noting all changes.
EC.1.3	The Developer must provide a properly formatted ECN notifying other teams of the changes.

5 PART II: TESTING REQUIRMENTS

Testing requirements T.1 through T.10 (including T.4.1 and T.9.1 through T.9.4) are unchanged. See project 1 Section 5. Testing requirements T.11 through T.16 are unchanged. See project 2 Section 5, Table 5. Table 7 lists the added testing requirements (10/15/2026).

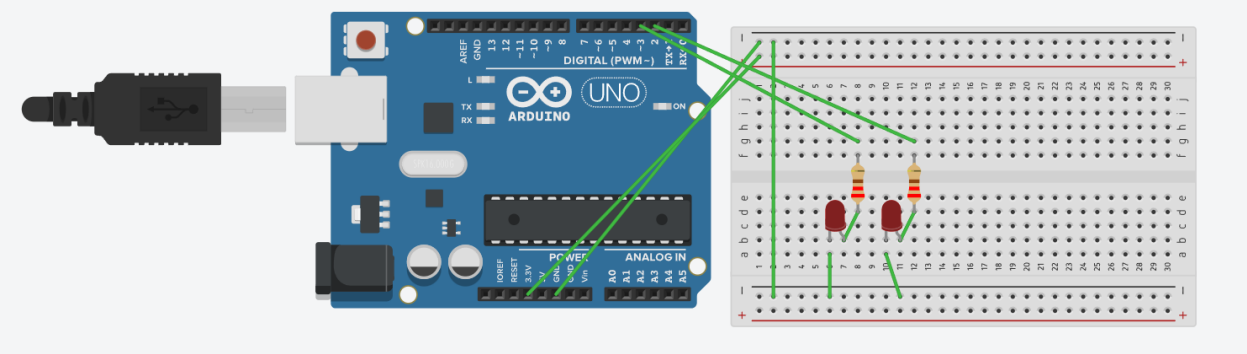
Table 7. Added testing requirements (new, 10/15/2026).

T.17	The Evaluator must inspect the source code and confirm that neither millis() nor delay() is called anywhere in the System, and that all timing is obtained from mymilli().
T.18	The Evaluator must inspect the source code and confirm that a function named mymilli() exists, takes no arguments, and returns the number of milliseconds elapsed since myMilliInit() was called as a 32-bit unsigned value.
T.19	The Evaluator must inspect the source code and confirm that the timer interrupt service routine (ISR) that drives mymilli() is written by the Developer and uses a PWM-capable timer/counter, and that it is the only timekeeping ISR the System depends on.
T.20	The Evaluator must inspect the source code and confirm that every register address and bit position used to configure the timer (and to save and restore the interrupt state) is defined by the Developer as a constant in the source, and that no register or bit name from avr/io.h (e.g. TCCR2A, OCR2A, TIMSK2, SREG, WGM21) is used.
T.21	The Evaluator must inspect mymilli() and confirm that the multi-byte counter is read with interrupts disabled and that the previous interrupt state is restored afterward.

T.22	The Evaluator must re-execute T.1 through T.16 (see project 1 and project 2) on the PROJECT-MYMILLI build and confirm the results are identical (regression test).
T.23	With one LED set to 1000 msec, the Evaluator must time at least 60 seconds of blinking using a video timestamp and confirm the average period is within ±1% of the entered interval.
T.24	Using a separate test sketch (a test fixture, not part of the System, which may call millis()), the Evaluator must print mymilli() and millis() together every 10 seconds for 60 seconds and confirm the two values never differ by more than 10 msec.
T.25	Using a separate test build in which myTicks is preset to 0xFFFFFFFF minus 5000, the Evaluator must confirm that an LED set to 1000 msec continues to blink at the correct rate through the 32-bit rollover.

6 PART III: DESIGN, BUILD AND TEST SYSTEM

Schematic:



Hardware Design:

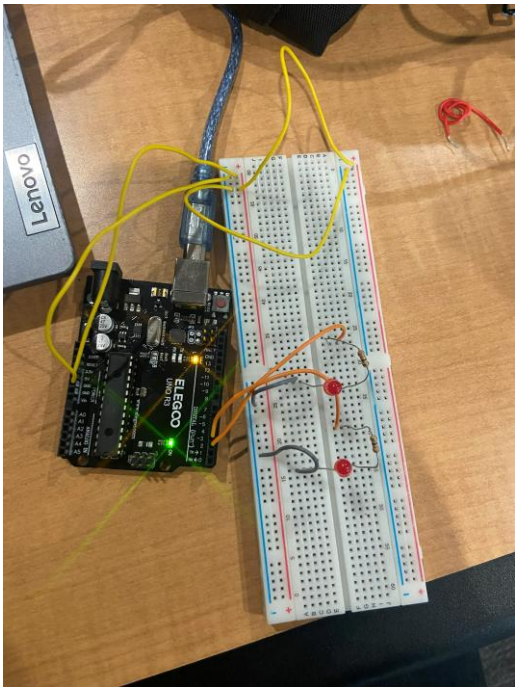


Table 8. Register constants defined by the Developer.

Constant	Address / bit	Purpose
MY_SREG	0x5F	Status register: save and restore the global interrupt state
MY_TIMSK2	0x70	Timer2 interrupt mask
MY_TCCR2A	0xB0	Timer2 control A (waveform mode)
MY_TCCR2B	0xB1	Timer2 control B (clock select)
MY_TCNT2	0xB2	Timer2 counter
MY_OCR2A	0xB3	Timer2 compare value A
MY_WGM21	bit 1	In MY_TCCR2A: CTC mode (count to MY_OCR2A, then restart)
MY_CS22	bit 2	In MY_TCCR2B: clock prescaler /64
MY_OCIE2A	bit 1	In MY_TIMSK2: compare-match A interrupt enable

mymilli(). mymilli() returns myTicks, the number of milliseconds since myMilliInit() was called. The AVR is an 8-bit processor, so reading a 32-bit value takes several instructions, and the ISR could change the value between bytes. mymilli() therefore saves the status register, disables interrupts, copies myTicks, and restores the status register. The status register is restored rather than interrupts being blindly re-enabled, so that mymilli() does not enable interrupts that its caller had disabled. The counter rolls over after 2^32 msec (about 49.7 days). Unsigned subtraction of two readings stays correct across the rollover as long as the interval measured is less than 2^31 msec.

Application changes. blink() and taskSerial() call mymilli() in place of millis(), and the interval and last-toggle variables are uint32_t to match the return type. myMilliInit() is called once from setup(). The dialog, pin assignments, 330 Ω resistors and task structure are unchanged.

Resources used. Timer2 is now used for timekeeping, so analogWrite() on Pins 3 and 11 is unavailable. The System uses digitalWrite() only, so there is no conflict. The serial port uses its own UART interrupts and is unaffected.

Table 9. Timer configuration.

Parameter	Value
CPU clock	16 MHz
Prescaler	64 (timer clock 250 kHz, 4 µs per count)
Compare value (OCR2A)	249 (250 counts per interrupt)
Interrupt period	250 × 4 µs = 1.000 msec
Counter	volatile uint32_t myTicks, incremented once per interrupt

Code:

```
#include <stdint.h>
#include <avr/interrupt.h>

//Own register constants
#define MY_SREG    (*(volatile uint8_t*)0x5F)
#define MY_TIMSK2  (*(volatile uint8_t*)0x70)
#define MY_TCCR2A  (*(volatile uint8_t*)0xB0)
#define MY_TCCR2B  (*(volatile uint8_t*)0xB1)
#define MY_TCNT2   (*(volatile uint8_t*)0xB2)
#define MY_OCR2A   (*(volatile uint8_t*)0xB3)

#define MY_WGM21   1
#define MY_CS22    2
#define MY_OCIE2A  1

// mymilli(): timer, ISR, and reader
volatile uint32_t myTicks = 0;

// fires every 1.000 ms
ISR(TIMER2_COMPA_vect) {
  myTicks++;
}

void myMilliInit() {
  MY_TCCR2A = (1 << MY_WGM21);
  MY_TCCR2B = (1 << MY_CS22);
  MY_OCR2A  = 249;
  MY_TCNT2  = 0;
  MY_TIMSK2 = (1 << MY_OCIE2A);
  __asm__ __volatile__("sei");
}

uint32_t mymilli() {
  uint8_t saved = MY_SREG;
  __asm__ __volatile__("cli");
  uint32_t copy = myTicks;
  MY_SREG = saved;
  return copy;
}

// Cyclic executive application from PROJECT2
typedef void (*Task)(void);
```

```
const int ledPin1 = 2;
const int ledPin2 = 3;

uint32_t interval1 = 0;
uint32_t interval2 = 0;
uint32_t lastToggle1 = 0;
uint32_t lastToggle2 = 0;
bool state1 = LOW;
bool state2 = LOW;

bool waitingForLed = true;
int  chosenLed = 0;
bool prompted = false;

char inBuf[12];
uint8_t inLen = 0;

// Non-blocking collects bytes already in the buffer, true when a line is complete
bool readLine() {
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n' || c == '\r') {
      if (inLen == 0) continue;
      inBuf[inLen] = '\0';
      inLen = 0;
      return true;
    }
    if (inLen < sizeof(inBuf) - 1) inBuf[inLen++] = c;
  }
  return false;
}

void blink(int pin, uint32_t interval, uint32_t &lastToggle, bool &state) {
  if (interval == 0) {
    if (state) { state = LOW; digitalWrite(pin, LOW); }
    return;
  }
  if (mymilli() - lastToggle >= interval) {
    state = !state;
    digitalWrite(pin, state);
    lastToggle += interval;
  }
```

```
}

void taskSerial() {
  if (!prompted) {
    Serial.println(waitingForLed ? F("What LED? (1 or 2)")
                                 : F("What interval (in msec)?"));
    prompted = true;
  }
  if (!readLine()) return;

  long v = atol(inBuf);
  if (waitingForLed) {
    if (v == 1 || v == 2) { chosenLed = v; waitingForLed = false; }
  } else {
    uint32_t half = (v > 1) ? v / 2 : 0;
    if (chosenLed == 1) { interval1 = half; lastToggle1 = mymilli(); }
    else                { interval2 = half; lastToggle2 = mymilli(); }
    waitingForLed = true;
  }
  prompted = false;
}

void taskLed1() { blink(ledPin1, interval1, lastToggle1, state1); }
void taskLed2() { blink(ledPin2, interval2, lastToggle2, state2); }

Task taskTable[] = { taskSerial, taskLed1, taskLed2 };
const uint8_t NUM_TASKS = sizeof(taskTable) / sizeof(taskTable[0]);

void setup() {
  Serial.begin(9600);
  pinMode(ledPin1, OUTPUT);
  pinMode(ledPin2, OUTPUT);
  myMilliInit();
}

void loop() {
  for (uint8_t i = 0; i < NUM_TASKS; i++) taskTable[i]();
}
```

Test Results: Table 10 records the results of the testing requirements on the PROJECT-MYMILLI build.

Table 10. Test results.

Test	Origin	Pass / Fail	Notes / measured values

T.1	Retained, project 1		
T.2	Retained, project 1		
T.3	Retained, project 1		
T.4	Retained, project 1		
T.5	Retained, project 1		
T.6	Retained, project 1		
T.7	Retained, project 1		
T.8	Retained, project 1		
T.9	Retained, project 1		
T.10	Retained, project 1		
T.11	Retained, project 2		
T.12	Retained, project 2		
T.13	Retained, project 2		
T.14	Retained, project 2		
T.15	Retained, project 2		
T.16	Retained, project 2		
T.17	New		
T.18	New		
T.19	New		
T.20	New		
T.21	New		
T.22	New		
T.23	New		
T.24	New		
T.25	New		

7 CONCLUSIONS

This project revised the design of project 2 so that timekeeping comes from a replacement for millis(), mymilli(), driven by a Developer-written Timer2 compare-match ISR and Developer-defined register constants, per ECO 0.3. The Customer Requirements C1 through C6 and High-level Requirements HL.1 through HL.6 are unchanged. The cyclic executive, serial dialog and LED tasks of project 2 are unchanged apart from calling mymilli(). Testing requirements T.1 through T.16 were retained and T.17 through T.25 were added. [COMPLETE AFTER TESTING:

state the results of Table 10, e.g. whether all tests passed, the measured period error from T.23, and the largest difference found in T.24.] The hardware, including the 330 Ω resistors, is unchanged from project 1.

REFERENCES:

project 1 Reference document woode.cmpe311.project#1, Project 1 (PROJECT-ASYNC), 09/22/2026.
project 2 Reference document woode.cmpe311.project#2, Project 2 (PROJECT-CE), including ECN1.1, 09/29/2026.

8 ENGINEERING CHANGE NOTE ECN2.1

PROJECT	PROJECT-CE (Project #2)
ORIGINATION	Prof. Kidd, ECO 0.3
PURPOSE OF CHANGE	Replace millis() with mymilli()
DOCUMENT NUMBER	ECN2.1
DATE OF PUBLICATION	10/15/2026
LEAD ENGINEER	Kid Kidd, UMBC
TEAM LEAD	Paa Woode
EXECUTIVE SUMMARY	Per ECO 0.3, the Arduino millis() function used by the design of project 2 was replaced by mymilli(), driven by a Developer-written Timer2 compare-match ISR that increments a 32-bit millisecond counter every 1.000 msec, with Developer-defined register constants. The cyclic executive, serial dialog and LED tasks are otherwise unchanged. Customer requirements C1 through C6 and high-level requirements HL.1 through HL.6 are unchanged. Testing requirements T.1 through T.16 are retained and T.17 through T.25 are added. Hardware is unchanged.
HIGH-LEVEL DESCRIPTION	The Arduino millis() function and its underlying timer, register definitions and ISR are replaced with a Developer-written equivalent, mymilli().
REASON FOR CHANGE	Gives the design team a function it can customize and extend, isolates the design from future changes to the third-party arduino.h, and avoids possible future IP ownership claims (ECO 0.3).
CHANGED DOCUMENTS	10/15/2026, project 2 Section 6, Software: ~~blink() and taskSerial() call millis(); timing variables are unsigned long~~ blink () and taskSerial() call mymilli(); timing variables are uint32_t. 10/15/2026, project 2 Section 4: educational constraints EC.1.0 through EC.1.3 of ECO 0.3 added; EC.1 through EC.5 of project 1 unchanged. 10/15/2026, project 2 Section 5: T.17 through T.25 added. 10/15/2026, project 2 Section 7: ~~"Testing requirements T.1 through T.10 were retained and T.11 through T.16 were added."~~ revised to cover T.1 through T.25 (see Section 7 of this document).
ADDED DOCUMENTS	This revision (ECN2.1 included); source file with myMilliInit(), mymilli(), the Timer2 ISR and the register constants; tests T.17 through T.25; Tables 8 and 9.
REMOVED DOCUMENTS	None. The System no longer calls millis() or delay().
REPLACED DOCUMENTS	The timekeeping portion of the software design of project 2 is replaced by Section 6 of this revision.

ANY ADDITIONAL DESIGN CONSIDERATIONS	Timer2 is used for timekeeping. analogWrite() on Pins 3 and 11 is unavailable (not used by the System). The Arduino core still initializes and runs Timer0 for its own millis(). The System does not call it. The Timer0 overflow ISR is not redefined, because the core already defines it. EC.1.0.1 is interpreted as an ISR on a PWM-capable timer/counter (Timer2). The ISR() macro and the vector name TIMER2_COMPA_vect come from avr/interrupt.h. All register addresses and bit positions are defined in the source (Table 8). AVR gcc from Arduino IDE 2.3.3 or later (EC.1 of project 1). No 3rd-party multitasking libraries (EC.5 of project 1). LEDs remain on Pins 2 and 3. Resistors remain 330 Ω per project 1.